Parallel Quick-Sort, Insertion-Sort Combo & Load Balancing Project Spring 2026

ECE/CS 566

Shantanu Dutt

Problem Description

- Quick sort (QS) based pivoting for partitioning the initial N numbers into P (# procs) subarrays. The initial set of N numbers are only available in proc. 0, and the required P-1 pivots and passing of the resulting subarrays to the other P-1 proc. can be done either sequentially by proc. 0 or in parallel with other processors. Let A(i) be the subarray in proc i after the above pivotings.
- Note that after P-1 pivots, the P subarrays are ordered in <= order: subarray j (in proc. j) #s <= subarray j+1 (in proc. j+1) numbers for 0 <= j <= P-2. This ordering is called the proc subarray ordering requirement (PSOR) and needs to be satisfied when the parallel computation finishes.
- As far as load-balancing (LB) is concerned, the above requirement can be satisfied in any of the following 2 ways:
  - LB occurs only between adjacent #'ed procs (e.g., i w/ i-1 and i+1), in which i gives its lowest k #s to i-1 and its highest k #s to i+1 (can be done in $\Theta(n(i)\log k)$ time using a k-element heap), where n(i) = |A(i)|, and k is a pre-determined or dynamically changing value. Note that this constrained LB space can significantly limit LB efficacy or speed across the P processors.
  - LB can occur between any 2 processors w/ any chunk of the donor proc's subarray transferred, but the receiving proc cannot merge the transferred subarray after sorting w/ its original subarray (as this will violate PSOR) but needs to do something else with it.
- The local sorting needs to be ONLY using *insertion sort* whose complexity for sorting n numbers can range from $\Theta(n)$ to $\Theta(n^2)$, depending upon "the degree of existing sorting" in the original array; see https://en.wikipedia.org/wiki/Insertion_sort. Thus, the computation load CL(A) for an array A of n numbers is input dependent, and an approximate estimate of this load relative to any other array needs to be made for the purpose of LB. Further, such an estimate should take at most O(n) time. From this point of view, one can either view the quantitative load of a array A of n numbers as n and load quality as CL(A), or only a qualitative load of CL(A)—note though that this will only be an estimate.

Problem Description (Cont'd)

- At the end of parallel sorting, each proc. needs to have a sorted subarray w/ the aforementioned PSOR between the subarrays of all processors
- Need to also determine when everyone has terminated, and after this, via sequential token passing in order of increasing proc. id, each processor (when it has the token) writes out its sorted subarray in a common file (labeling will be specified)
- Also to be written out by proc. 0 for each problem input instance and each P to another file (stat file) are:
  a) parallel times w/o LB and w/ or w/o initial pivoting time
  b) parallel times w/ LB and w/ or w/o initial pivoting time
  c) the 2 corresponding speedups for each P
  d) the degree of (i) quantitative imbalance given by the following *normalized standard deviations* measuring relative load imbalance: (standard deviation among the initial n(i)s after P-1 pivotings)/(N/P)., and (ii) qualitative imbalance given by: (standard deviation among the initial CL(A(i)s after P-1 pivotings)/(average of CL(A(i)s)
- If you have m > 1 LB algorithms designed, implemented and tested (extra credit), replace the b)-c) above with: "b) 2m parallel times for each of the m LB schemes and w/ or w/o initial pivoting time; c) the 2m corresponding speedups for each P"
- Relate the aforementioned relative load imbalances to the speedup and efficiency obtained for each P and derive insightful conclusions from these correlations.
- Finally, need:
  - A well written and scientifically high-quality report with insightful conclusions based on theoretical analysis and the obtained empirical data.
  - A similarly well-prepared presentation.